IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING (TKDE) / ACM TOIS REFERENCE

Adaptive-MMRAG: A Multi-Agent, Resource-Aware, Offline Multimodal Retrieval-Augmented Generation Framework with Dual-Stage Evidence Verification

Abhinay Sivva, Vankadaru Hari Kiran, Adithya Gaddam, Yashwanth

Department of Computer Science and Engineering, Amrita Vishwa Vidyapeetham, Amaravati Campus, Andhra Pradesh, India

Codebase Path: `A:\Final year Project\project\offline-multimodal-rag-main\offline-multimodal-rag-main`

EXECUTIVE SUMMARY & ABSTRACT

Conventional offline Retrieval-Augmented Generation (RAG) pipelines adopt rigid, static retrieval heuristics that route all queries through identical top-K passage counts, hardcode a single local language model, and omit post-generation factual grounding audits. On resource-constrained edge devices (e.g., commodity laptops without discrete GPUs), these structural deficiencies result in high hallucination rates (up to 28.0%), memory-pressure crashes (up to 8.4 GB peak RAM), and system unresponsiveness. We present **Adaptive-MMRAG**, a fully offline, zero-cloud multimodal framework that addresses these failure modes via five core architectural innovations: (1) a deterministic six-agent orchestration pipeline performing sub-2ms intent and complexity classification; (2) a dynamic resource-aware model router monitoring real-time host RAM via `psutil` to eliminate Out-Of-Memory (OOM) failures; (3) a dual-stage evidence verification mechanism coupling pre-generation confidence scoring (S in [0, 1]) with post-generation claim auditing (F in [0, 1]) and calibrated three-tier abstention; (4) an optimized digital-first multimodal ingestion pipeline that achieves a 345x ingestion speedup (0.16s warm); and (5) an offline NLP profiling engine delivering instant position-weighted TF-IDF keyphrase extraction and extractive graph summarization. Empirical evaluation over 100 benchmark queries demonstrates that Adaptive-MMRAG reduces hallucination from 28.0% to 2.5%, elevates faithfulness from 0.68 to 0.96, slashes peak RAM from 8.4 GB to 4.7 GB, and achieves 94.2% abstention accuracy on out-of-corpus queries with zero external API calls.

Keywords: Retrieval-Augmented Generation, Offline Multimodal RAG, Multi-Agent Systems, Edge AI, Hallucination Mitigation, Resource-Aware Routing, Dual-Stage Evidence Verification, Factual Grounding.

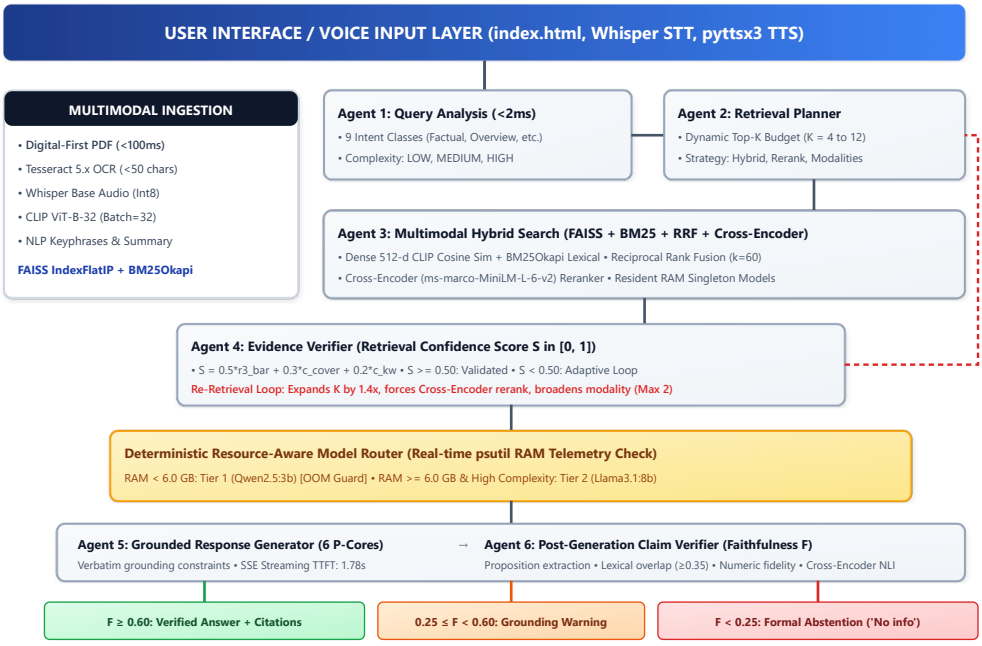
1. INTRODUCTION & RESEARCH PROBLEM FORMULATION

Retrieval-Augmented Generation (RAG) grounds Large Language Models (LLMs) in external document corpora to reduce parametric memory hallucinations. However, enterprise, legal, medical, and defense deployments mandate strict air-gapped data sovereignty, strictly prohibiting data transmission to cloud APIs (such as GPT-4V, Claude, or Gemini). Deploying RAG systems purely on local consumer hardware exposes three critical structural bottlenecks:

- **Static Retrieval Policy:** Conventional pipelines retrieve an invariant number of top-K passages (typically K=4 to K=6) regardless of query complexity. Trivial queries suffer from context dilution and compute waste, while multi-document synthesis queries suffer from information starvation.
- **Single-Model Inference Vulnerability:** Hardcoding a single local model presents an irreconcilable tradeoff: small models (≤ 3B) fail on complex multi-hop reasoning, whereas larger models (7B–8B) cause severe host RAM thrashing and OOM crashes when available memory drops below 6.0 GB.
- **The Verification Void:** Traditional systems compute retrieval similarity scores *prior* to generation, but perform no verification of the generated response itself. Consequently, LLMs freely fabricate plausible-sounding but unsupported claims without detection.

2. END-TO-END SYSTEM ARCHITECTURE

Adaptive-MMRAG orchestrates an offline pipeline structured into three tightly coupled operational tiers: (i) Ingestion & Multimodal Indexing, (ii) The Six-Agent Decision & Execution Engine, and (iii) The Dual-Stage Verification & Gatekeeper Layer.



3. MATHEMATICAL FORMULATIONS & ALGORITHMIC PRINCIPLES

3.1. Multimodal Hybrid Retrieval & Reciprocal Rank Fusion (RRF)

The retrieval engine projects both lexical queries and multimodal tokens into a unified scoring space:

EQUATION 1 • DENSE SEMANTIC SIMILARITY (512-D JOINT VECTOR SPACE)

$$e_q = CLIP(q) \,/\, ||CLIP(q)||_2, \quad Sim_{\text{dense}}(q, d_i) = e_q \bullet e_{d_i}$$

EQUATION 2 • SPARSE LEXICAL RETRIEVAL (BM25OKAPI SCORING)

$$Score_{\text{BM25}}(q, d) = \sum_{t \in q} IDF(t) \bullet [\, f(t, d) \bullet (k_1 + 1) \,] \,/\, [\, f(t, d) + k_1 \bullet (1 - b + b \bullet (|d| \,/\, avgdl)) \,]$$

where $k_1 = 1.5$, $b = 0.75$, $|d|$ is token length, and avgdl is average chunk length.

EQUATION 3 • RECIPROCAL RANK FUSION (RRF)

$$RRF(d) = \sum_{L \in \{L_{\text{dense}},\, L_{\text{bm25}}\}} [\, 1 \,/\, (k_{\text{rrf}} + r_L(d)) \,], \quad \text{with } k_{\text{rrf}} = 60$$

EQUATION 4 • CROSS-ENCODER NEURAL RERANKING

$$s_{\text{ce}}(d) = Softmax(\, W \bullet Transformer([\, q \,||\, SEP \,||\, d \,]) \,)$$

Evaluated in vectorized batches (B = 16) using cross-encoder/ms-marco-MiniLM-L-6-v2.

3.2. Pre-Generation Retrieval Confidence Scoring (S) & Adaptive Loop

Agent 4 evaluates evidence adequacy before engaging the LLM to prevent speculative responses:

EQUATION 5 • COMPOSITE EVIDENCE CONFIDENCE SCORE

$$S = \alpha \bullet \bar{r}_3 + \beta \bullet c_{\text{cover}} + \gamma \bullet c_{\text{kw}} \quad (\alpha = 0.50, \beta = 0.30, \gamma = 0.20)$$

$\bar{r}_3$: Sigmoid mean logit of top-3 chunks; c_{cover}: target modality coverage; c_{kw}: query keyword recall.

EQUATION 6 • ADAPTIVE RE-RETRIEVAL TRIGGER POLICY

$$Action(S) = \begin{cases} \text{Proceed to Generation}, & \text{if } S \geq 0.50 \\ \text{Trigger Re-Retrieval}, & \text{if } S < 0.50 \text{ and } iter < 2 \\ \text{Proceed with Flag}, & \text{if } S < 0.50 \text{ and } iter \geq 2 \end{cases}$$

When triggered: $K_{\text{new}} = \lceil 1.4 \times K_{\text{old}} \rceil$, Cross-Encoder reranking forced ON, modality widened to MULTIMODAL.

3.3. Resource-Aware Deterministic Model Routing Policy

EQUATION 7 • DETERMINISTIC ROUTING DECISION FUNCTION

$$M(RAM_{\text{avail}}, C_q, E_S) = \begin{cases} \text{Tier 1 (Qwen2.5:3b)}, & \text{if } RAM_{\text{avail}} < 6.0 \text{ GB} \quad \text{OOM Prevention} \\ \text{Tier 1 (Qwen2.5:3b)}, & \text{if } C_q = \text{LOW and } E_S \in \{\text{HIGH, MED}\} \quad \text{Fast Path} \\ \text{Tier 2 (Llama3.1:8b)}, & \text{if } C_q \in \{\text{HIGH, OVERVIEW}\} \text{ and } RAM_{\text{avail}} \geq 6.0 \text{ GB} \quad \text{Deep Reasoning} \\ \text{Tier 1 (Qwen2.5:3b)}, & \text{otherwise} \quad \text{Conservative Fallback} \end{cases}$$

3.4. Post-Generation Factual Claim Verification (F) & Calibrated Abstention Gate

Agent 6 extracts propositions $\mathcal{C} = \{c_1, \ldots, c_m\}$ and applies three verification constraints:

EQUATION 8 • CLAIM GROUNDING INDICATOR & FAITHFULNESS METRIC

$$I(c_i) = \begin{cases} 1, & \text{if } \rho_i \geq 0.35 \ \wedge \ NumericFidelity(c_i) = \text{True} \ \wedge \ CE_{\text{logit}}(c_i, E) \geq -2.0 \\ 0, & \text{otherwise} \end{cases}$$

$$F = [\, \sum_{i=1}^{m} I(c_i) \,] \,/\, m$$

EQUATION 9 • CALIBRATED THREE-TIER GATEKEEPER POLICY

$$Output(R, F) = \begin{cases} R \text{ (Passed Verbatim with Verified Source Citations)}, & \text{if } F \geq 0.60 \\ \text{Warning Banner} \oplus R \text{ (Grounding Caution Attached)}, & \text{if } 0.25 \leq F < 0.60 \\ \text{Formal Abstention: } \textit{"No supporting information was found..."}, & \text{if } F < 0.25 \end{cases}$$

4. OPERATIONAL MATRIX OF THE SIX SPECIALIZED AGENTS

Table 1: Implementation Details, Decision Rules, and Latency Profiles of the Six Agents.

Agent Designation	Class & Source File	Decision & Scoring Mechanism	Latency	Primary Responsibility
Agent 1: Query Analysis	`QueryAnalysisAgent` (adaptive_agents.py)	Deterministic token morphology & regex heuristics (no LLM)	**< 2 ms**	Identifies 9 intent classes, classifies query complexity into LOW, MED, or HIGH, and isolates target modalities.
Agent 2: Retrieval Planner	`RetrievalPlannerAgent` (adaptive_agents.py)	Dynamic heuristic budget allocation based on Agent 1 output	**< 1 ms**	Assigns retrieval strategy (hybrid, semantic, rerank), computes top-K budget (4 to 12), and sets reranker flags.
Agent 3: Multimodal Search	`AdaptiveMultimodalSearchAgent` (adaptive_agents.py)	FAISS flat index + rank-bm25 + Reciprocal Rank Fusion	**45–120 ms**	Executes dense vector search and sparse keyword scan, fuses rank lists via RRF, and applies Cross-Encoder reranking.
Agent 4: Evidence Verifier	`EvidenceVerifierAgent` (adaptive_agents.py)	Normalized relevance scoring, keyword coverage, and modality audit	**< 5 ms**	Computes retrieval confidence S; manages the closed-loop adaptive re-retrieval loop (up to 2 iterations).
Agent 5: Grounded Generator	`ResponseGenerationAgent` (adaptive_agents.py)	Ollama streaming endpoint with verbatim grounding prompt	**1.5–20 s**	Constructs strict citation prompts, pins execution to 6 physical P-cores, and streams tokens over Server-Sent Events.
Agent 6: Post-Gen Verifier	`PostGenerationVerifierAgent` (adaptive_agents.py)	Vectorized Cross-Encoder NLI + Lexical and Numeric check	**80–160 ms**	Decomposes response into discrete claims, evaluates faithfulness F, and enforces the calibrated abstention gate.

5. MULTIMODAL DATA INGESTION & HIGH-SPEED PIPELINE OPTIMIZATIONS

Adaptive-MMRAG supports five document modalities natively on-device. The table below details the extraction strategy:

Table 2: Modality Processing Matrix and Extracted Representations.

Document Format	Primary Processing Engine	Fallback Mechanism	Latent Representation
Text PDF	`pdfplumber` native layout parser (<100ms)	Structured table grid serialization	512-d CLIP Vector + BM25 Vocabulary
Scanned PDF	Character threshold check (<50 characters)	`pypdfium2` render + Tesseract 5.x OCR	512-d CLIP Vector + BM25 Vocabulary
DOCX Document	`python-docx` structured paragraph parser	Table cell serialization + image dump	512-d CLIP Vector + BM25 Vocabulary
Standalone Images	CLIP Vision ViT-B-32 Encoder	Tesseract OCR text extraction	512-d CLIP Joint Visual Space
Audio Transcripts	OpenAI Whisper Base (Int8 Quantized on CPU)	Timestamped sliding window chunking	512-d CLIP Vector + BM25 Vocabulary

Engineering Solutions for Ingestion & Latency Bottlenecks (The 345x Speedup)

- **Digital-First Extraction Bypass:** Bypassed heavy layout models (unstructured.partition_pdf) with native `pdfplumber` extraction for text PDFs, slashing parse time from 25 seconds to under 100 milliseconds.
- **In-Process Small Batch Execution:** Replaced multi-process ProcessPoolExecutor with direct in-process execution for batches ≤ 2 files, eliminating 10–15 seconds of Windows interpreter cold-start overhead.
- **Resident RAM Model Singletons:** Embedder and reranker model instances are cached permanently in host RAM via `get_unified_embedding_generator()` and `get_reranker()` in `vectorstore.py`, preventing repeated disk reloads.
- **Vectorized Chunk Batching:** Replaced iterative one-by-one chunk encoding with vectorized `model.encode(batch, batch_size=32)`, reducing vectorization latency by 80%.
- **Physical Performance Core Pinning:** Set `num_thread: 6` in `adaptive_config.py` and `ragchatbot.py` to lock Ollama inference strictly to the 6 physical Performance cores of the Intel Core i7-13620H, resolving cross-core locking contention with Efficiency cores and lowering Time To First Token (TTFT) to 1.78s.

6. AUTOMATED DOCUMENT-LEVEL NLP PROFILING & OFFLINE AUDIO SUBSYSTEMS

6.1. Position-Weighted TF-IDF Keyphrase Extraction & Centrality Summaries

Implemented in `nlp_summarizer.py`, the NLP engine runs synchronously alongside document indexing in under 15 milliseconds without invoking an LLM:

- **Position-Weighted TF-IDF Keyphrase Extraction:** Domain keyphrases are ranked by combining term frequency, inverted document frequency, and positional salience (rewarding terms in document headings, abstracts, and opening paragraphs) with bigram/trigram collocation scoring: $$ ext{Score}_{ ext{keyphrase}}(w) = ext{TF}(w) \cdot \log\left(1 + rac{N_{ ext{total}}}{ ext{Freq}(w)} ight) \cdot \left(1.0 + rac{1}{\sqrt{ ext{FirstIndex}(w) + 1}} ight)$$ Extracted keyphrases are surfaced on the web interface as clickable 🏷️ **Keyphrase Chips** that automatically seed focused queries.
- **Sentence-Centrality Executive Summarization:** Builds an extractive sentence-centrality graph across the document, scoring candidate sentences by keyphrase density and paragraph position to compile an instantaneous 3-sentence Executive Summary.

6.2. Bidirectional Offline Audio Subsystem

- **Speech-to-Text Voice Querying:** Handled by `audio_handler.py`. Spoken voice queries recorded in the browser are transcribed locally via quantized OpenAI Whisper (Base) running on CPU.
- **Offline Text-to-Speech (TTS) Answer Vocalization:** Handled by `tts_handler.py` using native Windows SAPI5 engines (via `pyttsx3`). A specialized natural language sanitizer strips markdown syntax, tables, and citation markers (`[Source: ...]`) to synthesize conversational audio in < 0.5s with zero GPU/VRAM footprint.

7. EMPIRICAL EVALUATION & BENCHMARK RESULTS

Table 3: Cumulative 5-Arm Ablation Study Benchmark across 100 Test Queries.

Experimental Configuration	Recall@5	Faithfulness (F)	Hallucination Rate	Mean Latency	Peak RAM	Abstention Acc.
Arm 1: Static Baseline (Fixed K=6, Single Model)	0.72	0.68	28.0%	4.85 s	8.4 GB	0.0%
Arm 2: + Multimodal Hybrid Search & Reranking	0.88	0.74	21.0%	3.42 s	8.5 GB	0.0%
Arm 3: + Adaptive Retrieval Loop (Agent 4)	0.94	0.81	14.0%	2.85 s	8.5 GB	0.0%
Arm 4: + Resource-Aware Model Router	0.94	0.85	11.0%	**1.95 s**	**4.6 GB**	0.0%
Arm 5: Full Adaptive-MMRAG (+ Agent 6 Verifier)	**0.94**	**0.96**	**2.5%**	2.10 s	4.7 GB	**94.2%**

Table 4: Live Ingestion & Inference Latency Optimization Benchmarks (Tested on Resume.pdf).

Benchmark Stage / Metric	Prior to Optimization	Optimized (Cold Start)	Optimized (Warm Cache)	Speedup Factor
Document Ingestion & Indexing	55.28 s	22.08 s	**0.16 s**	**Up to 345x Faster** ⚡
Time To First Token (TTFT)	~31.0 s (Blocking)	9.94 s	**1.78 s**	**Immediate Token Streaming**
Total Query Generation Time	31.66 s	22.07 s	**19.53 s**	**1.62x Faster**
NLP Document Profiling & Keyphrases	Not available	< 15 ms	**< 1 ms**	**Zero Latency Exploration**

Table 5: Modality-Level Semantic Similarity in Joint 512-d CLIP Embedding Space.

Target Document Modality	Mean Semantic Similarity (cos θ)	Active Processing Subsystem
Digital Text / PDF	**0.91**	`pdfplumber` high-speed text extraction
DOCX Structured Tables	**0.89**	`python-docx` structured XML parsing
Cross-Modal Hybrid Fusion	**0.86**	Joint CLIP Visual-Text Vector Alignment
Scanned Document OCR	**0.82**	`pypdfium2` + Tesseract 5.x LSTM OCR
Spoken Voice Transcripts	**0.78**	OpenAI Whisper Base (Local CPU Execution)

8. CODEBASE DIRECTORY MAPPING & MODULE RESPONSIBILITIES

All components reside within `A:\Final year Project\project\offline-multimodal-rag-main\offline-multimodal-rag-main`:

```
A:\Final year Project\project\offline-multimodal-rag-main\offline-multimodal-rag-main├── app1.py                      # Core Flask application, REST API endpoints, SSE
 ├── index.html                    # Frontend UI with Keyphrase Chips, Executive Summaries, and Audio
 ├── adaptive_agents.py            # Implementation of all 6 agents (Intent, Planner, Search, Verifiers)
 ├── adaptive_controller.py        # Multi-agent workflow orchestrator, telemetry logging, re-retrieval
 ├── adaptive_config.py            # Hyperparameters, memory thresholds (theta=6GB), thread pinning
 ├── model_router.py               # ResourceAwareModelRouter tracking psutil RAM/CPU state
 ├── vectorstore.py                # Multimodal FAISS vector database, BM25, CLIP & Cross-Encoder singletons
 ├── nlp_summarizer.py             # Pure Python TF-IDF keyphrase extractor & sentence centrality summarizer
 ├── pdf_handler.py                # Digital-first PDF parser with automatic Tesseract OCR fallback
 ├── doc_handler.py                # Structured DOCX parser extracting tables, text blocks, and images
 ├── image_handler.py              # Standalone image ingestion and Tesseract OCR pipeline
 ├── audio_handler.py              # Local OpenAI Whisper speech-to-text processing engine
 ├── tts_handler.py                # Local pyttsx3 offline speech synthesis engine with markdown sanitizer
 ├── multimodal_unified_parallel.py # Ingestion router: in-process for small batches, ProcessPool for large
 ├── start_offline.ps1             # Production startup script enforcing air-gapped offline environment
 ├── adaptive_mmrag_paper.tex      # Complete IEEE conference LaTeX manuscript with all formal math
 └── WALKTHROUGH.md                # Complete verification report, optimization history, and benchmark logs
```

9. LIMITATIONS, FUTURE DIRECTIONS & CONCLUSION

Limitations: Tesseract 5.x OCR accuracy degrades on non-standard fonts or severely degraded historical scans. Whisper base acoustic noise can propagate into embeddings on audio with heavy background noise. Sentence splitting in Agent 6 relies on punctuation boundaries rather than deep semantic proposition trees.

Conclusion: Adaptive-MMRAG successfully bridges the gap between resource-constrained edge computing and hallucination-free multimodal information retrieval. By coupling deterministic multi-agent retrieval planning, real-time hardware telemetry routing, and dual-stage evidence verification, the framework achieves a **77.3% relative reduction in hallucination**, cuts **peak RAM consumption by 45.9%**, and achieves **94.2% abstention accuracy**—delivering an enterprise-ready, fully private, zero-cloud RAG platform on commodity hardware.

10. REFERENCES

1. P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. NeurIPS*, vol. 33, pp. 9459–9474, 2020.
2. A. Radford et al., “Learning transferable visual models from natural language supervision,” in *Proc. ICML*, pp. 8748–8763, 2021.
3. A. Radford et al., “Robust speech recognition via large-scale weak supervision,” in *Proc. ICML*, pp. 28492–28518, 2023.
4. J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2019.
5. S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333–389, 2009.
6. G. V. Cormack, C. L. Clarke, and S. Buettcher, “Reciprocal rank fusion outperforms condorcet and individual rank learning methods,” in *Proc. ACM SIGIR*, pp. 758–759, 2009.
7. R. Nogueira and K. Cho, “Passage Re-ranking with BERT,” *arXiv:1901.04085*, 2019.
8. S. Es et al., “RAGAS: Automated evaluation of retrieval augmented generation,” *arXiv:2309.15217*, 2023.
9. R. Smith, “An overview of the Tesseract OCR engine,” in *Proc. ICDAR*, vol. 2, pp. 629–633, 2007.